



Advanced NLP

Session 5 - Recurrent Sequence Architectures: RNNs, LSTMs & GRUs

Teacher: Dr. Behrouz Minaei-Bidgoli

Presenter: Alireza Nazari

Presentation creator: Alireza Nazari

Contents

1. Language Modeling: the foundation

2. Recurrent Neural Networks

3. Training RNN Language Models

4. Vanishing & Exploding Gradients

5. Gated Architectures: GRU & LSTM

6. Machine Translation: from SMT to NMT

1.

Language Modeling: the foundation

What is Language Modeling?

Core Definition

Language Modeling is the task of predicting what word comes next. A system that does this is called a **Language Model**. This is the most important concept in the course — it leads to most of modern NLP.

the students opened their

books

laptops

exams

More formally

Given a sequence of words $x^{(1)}, \dots, x^{(t)}$, compute the probability distribution of the next word $x^{(t+1)}$:

$$P(x^{(t+1)} \mid x^{(t)}, \dots, x^{(1)})$$

where $x^{(t+1)}$ can be any word in the vocabulary:

$$x^{(t+1)} \in V = \{w_1, \dots, w_{|V|}\}$$

A second view: a probability for any text

You can also think of a Language Model as a system that **assigns a probability to a piece of text**: multiply the next-word probabilities along the sequence.

$$\begin{aligned} P(x^{(1)}, \dots, x^{(T)}) &= \prod_{t=1}^T P(x^{(t)} \mid x^{(t-1)}, \dots, x^{(1)}) \\ &= P(x^{(1)}) \times P(x^{(2)} \mid x^{(1)}) \times \dots \times P(x^{(T)} \mid x^{(T-1)}, \dots, x^{(1)}) \end{aligned}$$

This is what our LM provides

Why should we care about Language Modeling?

01 A benchmark task

Language Modeling is a benchmark task that helps us measure our progress on predicting language use. A model that predicts the next word well must, implicitly, know grammar, facts and context.

02 A subcomponent of many NLP tasks

...especially those involving generating text or estimating the probability of text:

Predictive typing

Speech recognition

Handwriting recognition

Spelling / grammar correction

Authorship identification

Machine translation

Summarization

Dialogue

03 The road to today

1

n-gram LMs

count statistics of word chunks (pre-deep learning)

2

Neural LMs

fixed-window networks remove sparsity (Bengio 2003)

3

Recurrent LMs

process any length input, one step at a time (today)

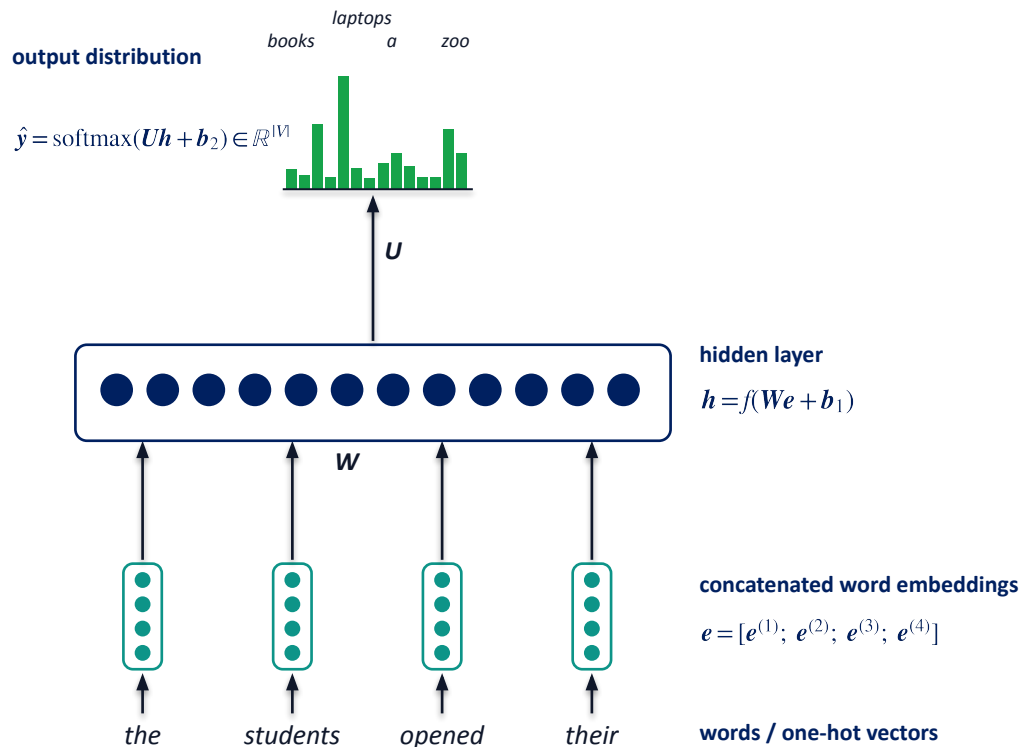
4

Transformer LMs

attention over the whole context — ChatGPT is an LM

Everything else in NLP has been rebuilt upon Language Modeling. Today we build the first neural LM that can read a whole sequence.

A fixed-window neural Language Model



Approximately: Y. Bengio et al. (2000/2003), "A Neural Probabilistic Language Model"

Improvements over n-gram LM

- No sparsity problem
- Don't need to store all observed n-grams — just the weights

Remaining problems

- Fixed window is too small
- Enlarging the window enlarges W — the window can never be large enough!
- $x_{(1)}$ and $x_{(2)}$ are multiplied by completely different weights in W : **no symmetry** in how inputs are processed

We need a neural architecture that can process
any length input

2.

Recurrent Neural Networks

Sequence models: three ways to condition on context

A sequence model maps a sequence to one representation per position:

$$f_{\theta}(x_1, \dots, x_{|x|}) \rightarrow h_1, \dots, h_{|x|}, \quad h_t \in \mathbb{R}^d$$

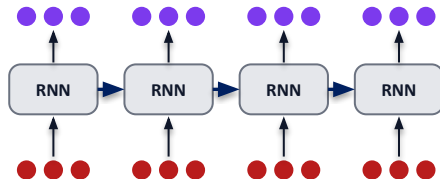
Our goal as a language model

$$P(X) \approx \prod_{t=1}^T p_{\theta}(x_t | x_1, \dots, x_{t-1})$$

next token | *full context — in theory, infinite*

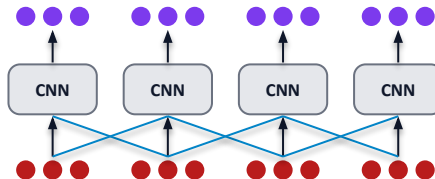
Recurrence

Condition representations on an encoding of the history. Information flows left-to-right through a hidden state.



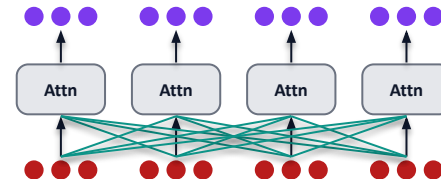
Convolution

Condition representations on local context — a fixed window of neighbours, like the fixed-window LM but stacked.



Attention

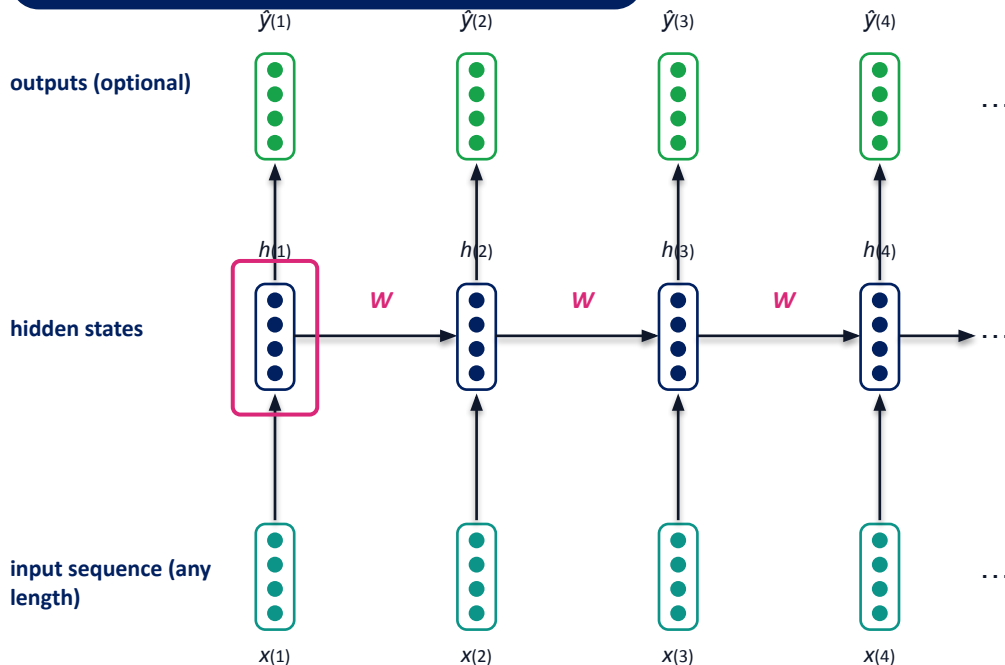
Condition representations on a weighted average of all tokens. Next lecture: attention & Transformers.



Recurrent Neural Networks: the core idea

Core idea: apply the same weights W repeatedly

A family of neural architectures for sequences of any length (Elman, 1990)



One update rule, reused at every step

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$$

σ : activation function (tanh, ReLU, ...)

$$W_h \in \mathbb{R}^{d \times d}, \quad W_x \in \mathbb{R}^{d \times d_{in}}, \quad b \in \mathbb{R}^d$$

- $h_t \in \mathbb{R}^d$ is the **hidden state** — intuitively a “memory” of everything seen so far
- The **same parameters** θ are used at every time step, so the model size does not grow with the sequence length

A simple RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax}(U\mathbf{h}^{(t)} + \mathbf{b}_2) \in \mathbb{R}^{|\mathcal{V}|}$$

hidden states

$$\mathbf{h}^{(t)} = \sigma(W_h \mathbf{h}^{(t-1)} + W_e \mathbf{e}^{(t)} + \mathbf{b}_1)$$

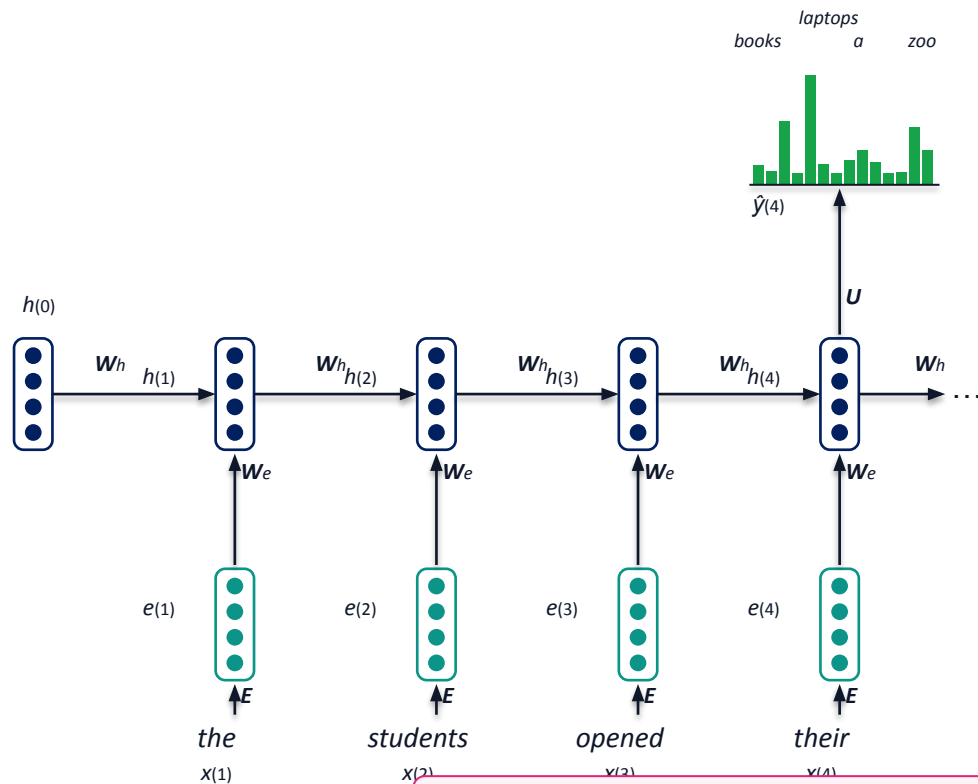
$\mathbf{h}^{(0)}$ is the initial hidden state (often zeros)

word embeddings

$$\mathbf{e}^{(t)} = E \mathbf{x}^{(t)}$$

words / one-hot vectors

$\mathbf{x}^{(t)} \in \mathbb{R}^{|\mathcal{V}|}$ — the same E , W_e , W_h , U at every step



Note: this input sequence could be much longer now!

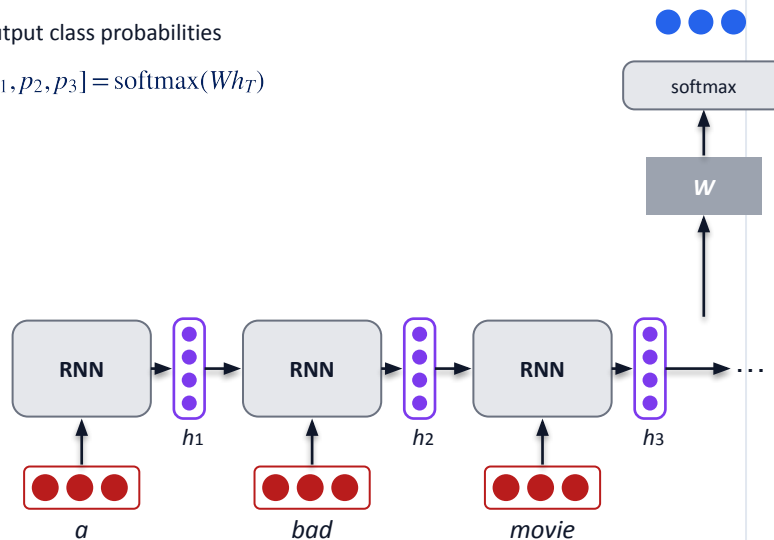
One machine, two uses: classification and language modeling

Example: sequence classification

Read the whole sequence, then classify from the last hidden state

Output class probabilities

$$[p_1, p_2, p_3] = \text{softmax}(Wh_T)$$

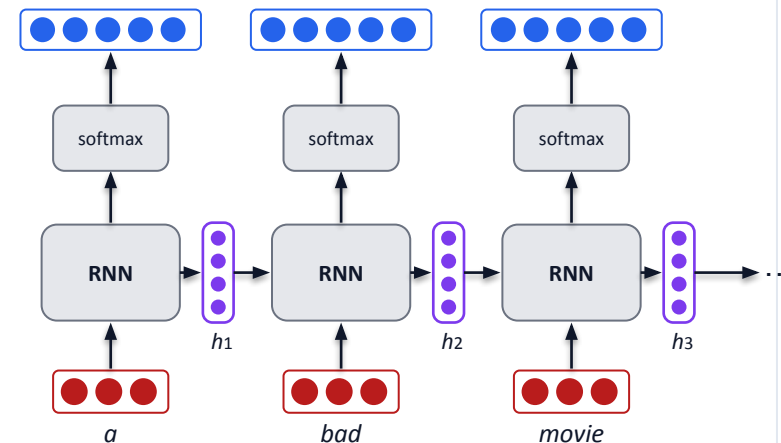


Example: language modeling

Predict the next token at every step

Next-token probabilities

$$p_{\theta}(\cdot | x_{<t}) = \text{softmax}(Wh_t)$$



Mikolov et al. 2010, Recurrent neural network based language model

RNN Language Models: advantages and disadvantages

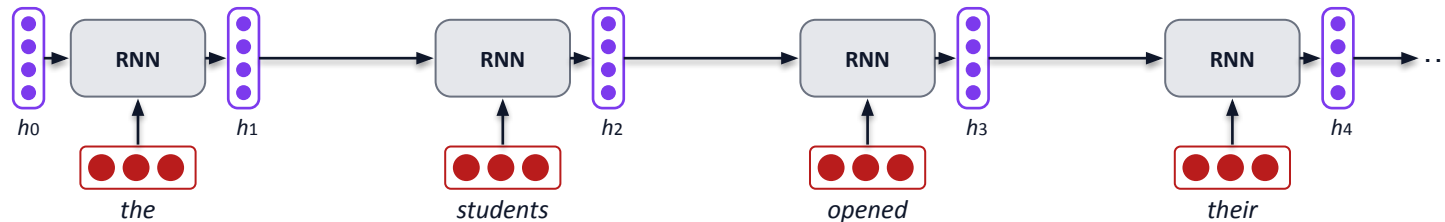
RNN advantages

- **Any length input.** The same cell is applied at every step; nothing is fixed to a window.
- **Long-range information (in theory).** Computation for step t can use information from many steps back.
- **Constant model size.** Model size doesn't increase for longer input context.
- **Symmetry.** Same weights applied on every timestep, so there is symmetry in how inputs are processed.

RNN disadvantages

- **Recurrent computation is slow.** Step t needs step $t-1$: training is hard to parallelize on GPUs.
- **Hard to access distant information in practice.** Gradients vanish, so long dependencies are not learned well.
- **Hidden state is a bottleneck.** Everything about the past must fit in one fixed-size vector.

More on these later: vanishing gradients (Part 4), attention (Part 6)



3.

Training RNN Language Models

Training an RNN Language Model

1

Get a big corpus of text

...which is a sequence of words $x_{(1)}, \dots, x_{(T)}$

2

Feed it into the RNN-LM

Compute the output distribution $\hat{y}_{(t)}$ for every step t — i.e., predict the probability distribution of every word, given the words so far.

3

Loss at step t : cross-entropy

...between the predicted distribution $\hat{y}_{(t)}$ and the true next word $y_{(t)}$ (a one-hot vector for $x_{(t+1)}$).

4

Average over all steps

...to get the overall loss for the entire training set.

Loss on step t

$$J^{(t)}(\theta) = CE(y^{(t)}, \hat{y}^{(t)}) = - \sum_{w \in V} y_w^{(t)} \log \hat{y}_w^{(t)} = -\log \hat{y}_{x_{t+1}}^{(t)}$$

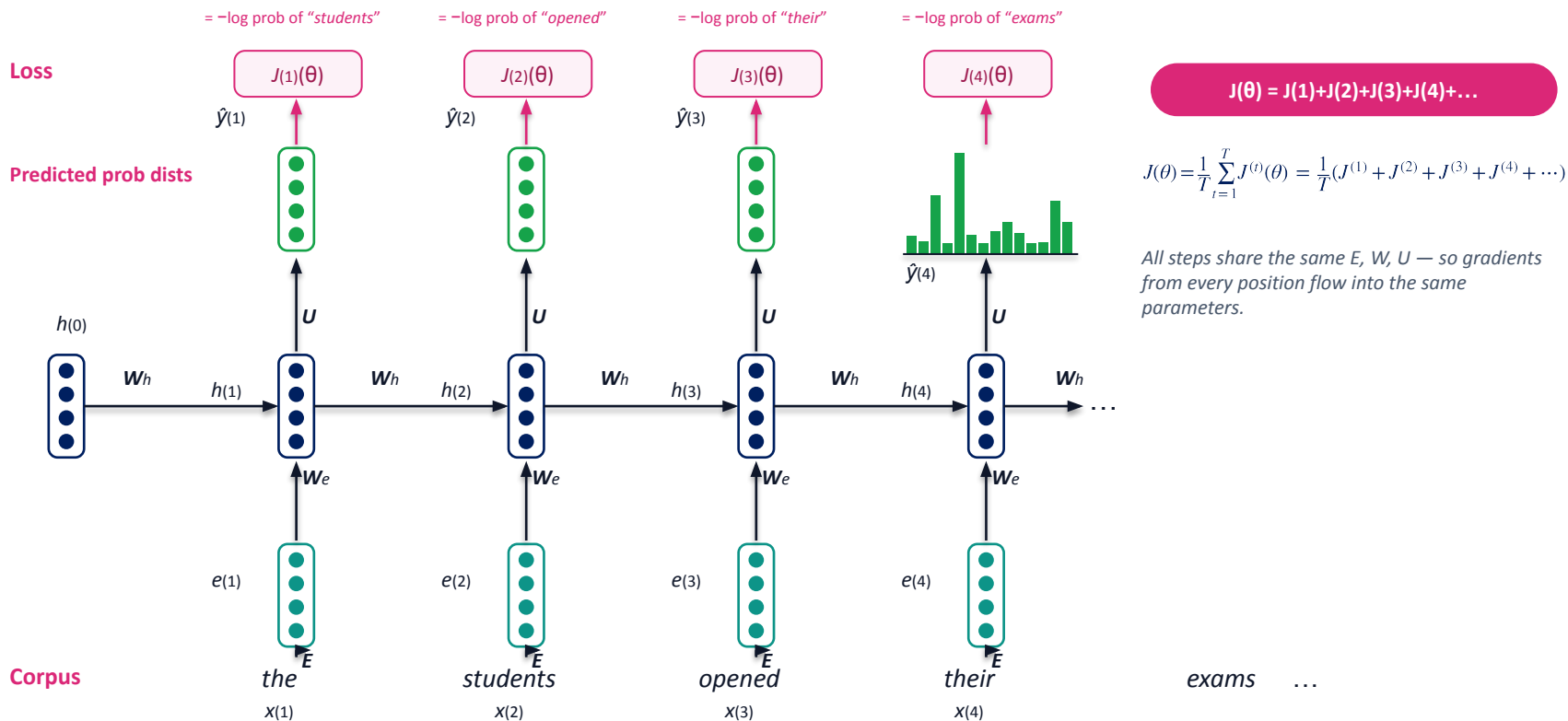
Only the true next word has $y_w = 1$, so the sum collapses to **minus the log-probability the model gave to the word that actually came next.**

Overall loss

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T -\log \hat{y}_{x_{t+1}}^{(t)}$$

Lower is better. In Part 3 we will see that **exp(J)** is exactly the perplexity of the model.

Training an RNN-LM: a loss at every step (“teacher forcing”)



Backpropagation Through Time (BPTT)

Question: What's the derivative of $J(t)(\theta)$ w.r.t. the repeated weight matrix W_h ?

Answer:

$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^t \frac{\partial J^{(i)}}{\partial W_h} \Big|_{(i)}$$

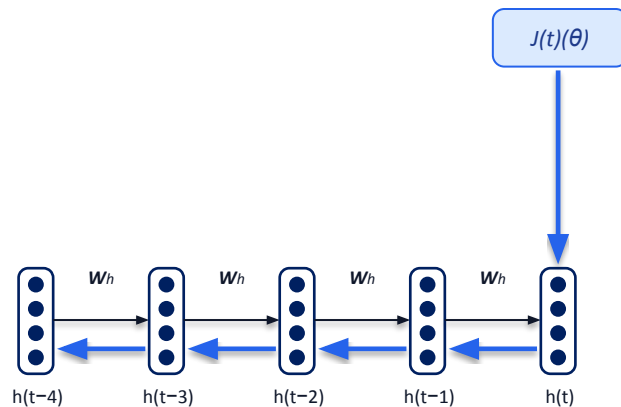
"The gradient w.r.t. a repeated weight is the sum of the gradient w.r.t. each time it appears."

Why? The multivariable chain rule

$$\frac{d}{dt} f(x(t), y(t)) = \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt}$$

Gradients **sum at outward branches**: W_h feeds every step, so every step contributes a term.

Backpropagate over timesteps $i = t, \dots, 0$, summing as you go



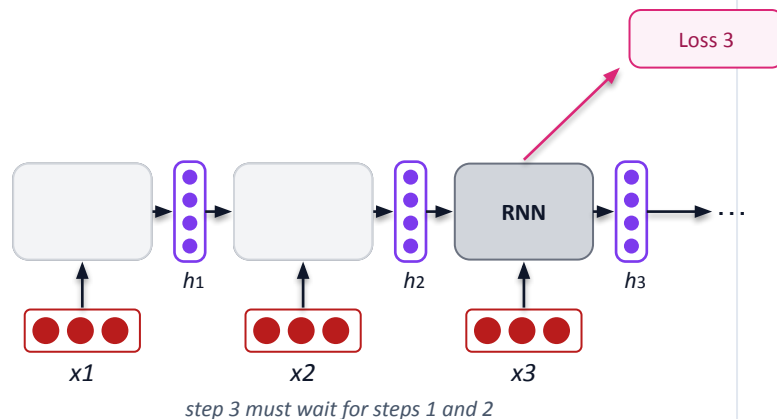
blue arrows = gradient flowing backwards through time

In practice, often "truncated" after ~20 timesteps for training-efficiency reasons. This algorithm is called "backpropagation through time" [Werbos, 1988].

The price of recurrence: parallelism and memory

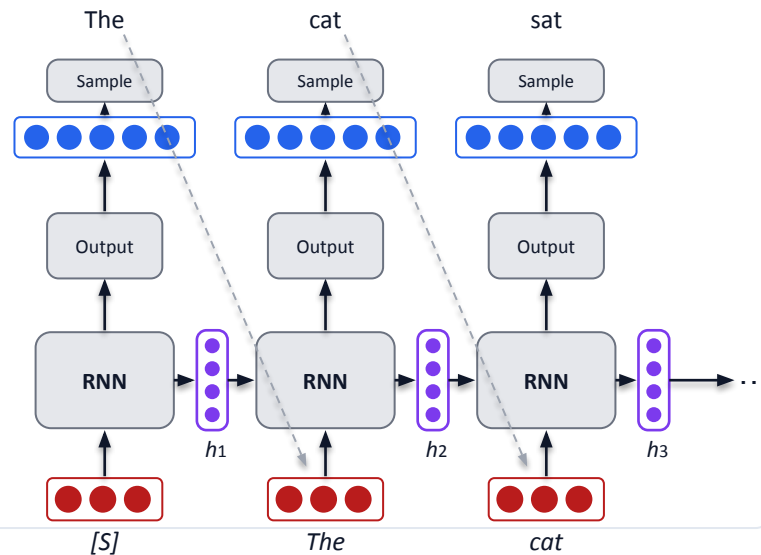
Training: hard to parallelize

- Computing the loss at step t requires the hidden state h_t
- Computing h_t requires h_{t-1} , h_{t-2} , ...
- As a result, RNN training is **difficult to parallelize** across time — a key reason Transformers took over



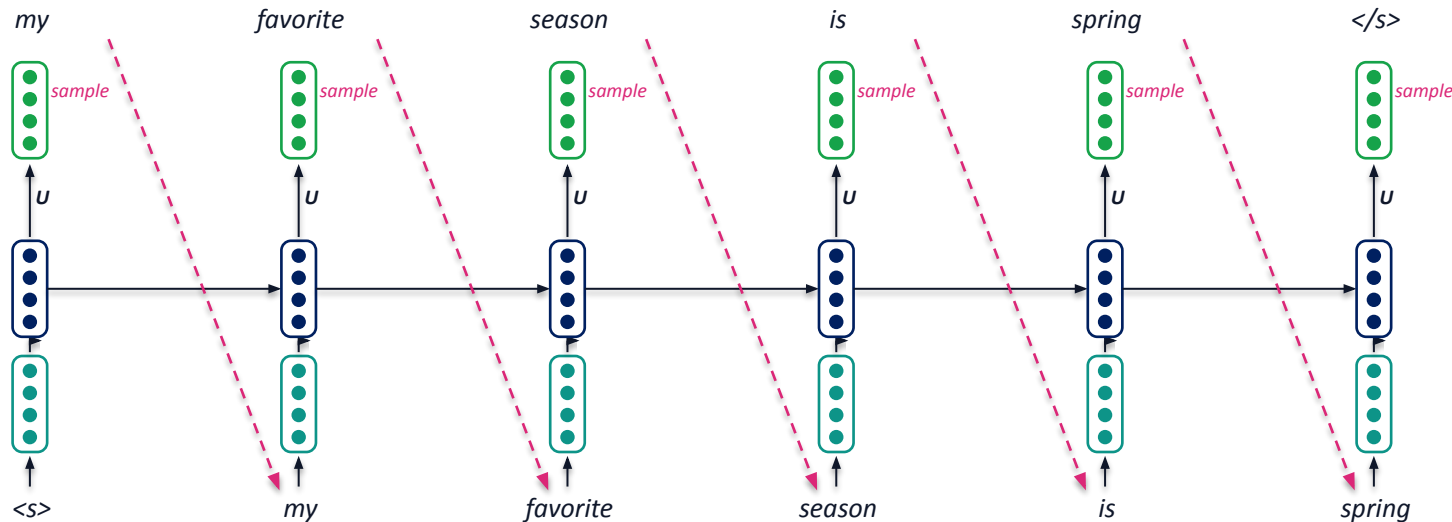
Inference: cheap and constant-memory

- Generate one token, use the new hidden state for the next step, repeat
- We only need to store the **previous hidden state**: **constant memory** as sequence length increases
- Each step is $O(1) \rightarrow O(T)$ for a length- T sequence



Generating text with an RNN-LM (“generating roll-outs”)

Just like an n-gram LM, you can use an RNN-LM to generate text by repeated sampling. The sampled output becomes the next step’s input.



Let’s have some fun!

You can train an RNN-LM on any kind of text, then generate text in that style.

RNN-LM trained on Harry Potter

“Sorry,” Harry shouted, panicking—“I’ll leave those brooms in London, are they?” “No idea,” said Nearly Headless Nick, casting low close to Cedric...

RNN-LM trained on recipes

Title: CHOCOLATE RANCH BARBECUE • 2 tb Parmesan cheese — chopped • 1 c Coconut milk • 3 Eggs, beaten • Place each pasta over layers of lumps.

Evaluating Language Models: perplexity

The standard **evaluation metric** for Language Models is **perplexity**.

(Supplementary intuition beyond the source slides)

$$\text{perplexity} = \prod_{t=1}^T \left(\frac{1}{P_{LM}(x^{(t+1)} | x^{(t)}, \dots, x^{(1)})} \right)^{1/T}$$

← Normalized by number of words (geometric mean)

Inverse probability of the corpus, according to the Language Model

This is equal to the exponential of the cross-entropy loss $J(\theta)$:

$$= \prod_{t=1}^T \left(\frac{1}{\hat{y}_{x_{t+1}}^{(t)}} \right)^{1/T} = \exp \left(\frac{1}{T} \sum_{t=1}^T -\log \hat{y}_{x_{t+1}}^{(t)} \right) = \exp(J(\theta))$$

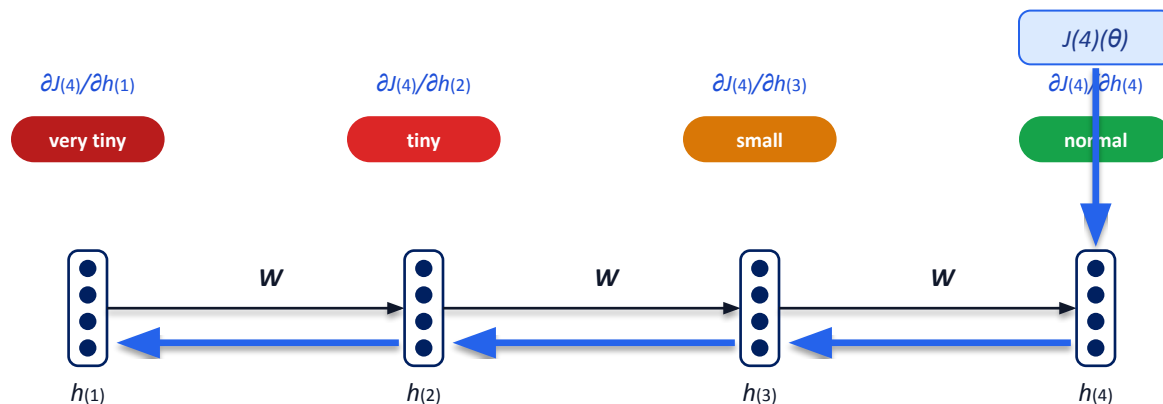
Lower perplexity is better!

Intuition: perplexity is the **effective branching factor** — a model with perplexity 50 is, on average, as uncertain as if it were choosing uniformly among 50 words at every step. Because $\exp(J)$ is monotone in J , training to minimize cross-entropy is the same as training to minimize perplexity.

4.

Problems with RNNs: Vanishing & Exploding Gradients

Vanishing gradient intuition



Chain rule! The gradient reaching an early state is a product of the Jacobians along the path:

$$\frac{\partial J^{(4)}}{\partial \mathbf{h}^{(1)}} = \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{h}^{(1)}} \times \frac{\partial \mathbf{h}^{(3)}}{\partial \mathbf{h}^{(2)}} \times \frac{\partial \mathbf{h}^{(4)}}{\partial \mathbf{h}^{(3)}} \times \frac{\partial J^{(4)}}{\partial \mathbf{h}^{(4)}}$$

What happens if these are small?

Source: "On the difficulty of training recurrent neural networks", Pascanu et al., 2013

Vanishing gradient problem: when these are small, the gradient signal gets smaller and smaller as it backpropagates further. **Implication: the model cannot learn long dependencies!**

Vanishing gradient: why? (the math)

Normal RNN: $h_t = \tanh(W_{in}x_t + Wh_{t-1}), \quad y_T = W_{out}h_T$

Total gradient w.r.t. W

$$\frac{\partial L}{\partial W} = \sum_{t=0}^T \frac{\partial L}{\partial y_T} \frac{\partial y_T}{\partial h_T} \frac{\partial h_T}{\partial h_t} \frac{\partial h_t}{\partial W}$$

Each time step t contributes a term that must travel from the loss at T back to step t .

The long-range factor

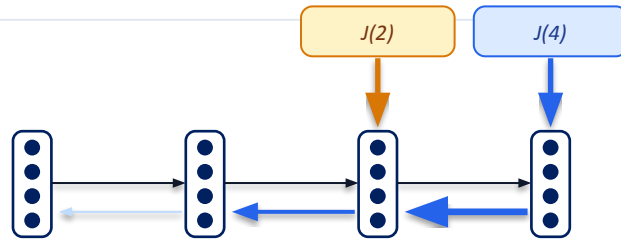
...is a product of $T-t$ Jacobians

$$\frac{\partial h_T}{\partial h_t} = \frac{\partial h_T}{\partial h_{T-1}} \frac{\partial h_{T-1}}{\partial h_{T-2}} \dots \frac{\partial h_{t+1}}{\partial h_t} = \prod_{t'=t}^{T-1} \frac{\partial h_{t'+1}}{\partial h_{t'}}$$

Why is vanishing gradient a problem?

Gradient signal from **far away** is lost because it's much smaller than gradient signal from **close-by**.

So, model weights are updated only with respect to **near effects**, not long-term effects.



$h_{(1)}$ receives a strong signal from the nearby loss $J_{(2)}$ but almost nothing from the far-away $J_{(4)}$: the network learns short-range patterns only.

Effect of vanishing gradient on an RNN-LM

LM task: When she tried to print her **tickets**, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____

- To learn from this training example, the RNN-LM needs to model the dependency between “**tickets**” on the 7th step and the target word “**tickets**” at the end.
- But if the gradient is small, the model **can't learn this dependency**
- So, the model is **unable to predict similar long-distance dependencies at test time**.

Exploding gradients — and gradient clipping

Why is exploding gradient a problem?

If the gradient becomes too big, then the SGD update step becomes too big:

$$\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} J(\theta)$$

learning rate

gradient

- This can cause **bad updates**: we take too large a step and reach a weird and bad parameter configuration (with large loss)
 - “You think you’ve found a hill to climb, but suddenly you’re in Iowa.”
- In the worst case, this will result in **Inf or NaN** in your network (then you have to restart training from an earlier checkpoint)

Gradient clipping: the solution

If the norm of the gradient is greater than some threshold, **scale it down** before applying the SGD update.

Algorithm 1 Pseudo-code for norm clipping

$$\hat{g} \leftarrow \frac{\partial E}{\partial \theta}; \quad \text{if } \|\hat{g}\| \geq \text{threshold}: \quad \hat{g} \leftarrow \frac{\text{threshold}}{\|\hat{g}\|} \hat{g}$$

- **Intuition**: take a step in the **same direction**, but a **smaller step**
- In practice, **remembering to clip gradients is important**, but exploding gradients are an **easy problem to solve**
- **Vanishing gradients are the hard problem — they need a change of architecture**

How to fix the vanishing gradient problem?

The main problem is that it's *too difficult for the RNN to learn to preserve information over many timesteps*.

In a vanilla RNN the hidden state is constantly rewritten

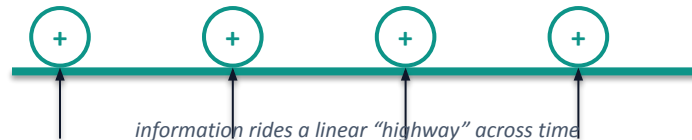
$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_x x^{(t)} + b)$$



each step overwrites the whole state through a nonlinearity

First idea: a separate memory which is added to

Long Short-Term Memory (LSTM) — a cell state c_t that is updated additively, with gates deciding what to keep.



information rides a linear “highway” across time

And then: creating more direct and linear pass-through connections in the model

Gating (GRU / LSTM)

learned gates interpolate between keeping and rewriting the state — next section

Residual connections

$h_t = h_{t-1} + \tilde{h}_t$: the gradient of the identity path is exactly 1

Attention

every output can look directly at every input — no path through many steps (Part 6, and Session 6)

5.

Gated Architectures: GRU & LSTM

A solution: gating and additive connections

Basic idea: pass information across timesteps with a learned “gate”

$$z_t = \sigma(W_z x_t + W_z h_{t-1})$$

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t$$

keep part of the old state + write part of the new one

Candidate new hidden state, e.g. a normal RNN update:

$$\tilde{h}_t = \tanh(W_x x_t + W_h h_{t-1})$$

To retain a long-term dependency, the model can set $z \rightarrow 0$ for multiple steps:

$$\frac{\partial h_{t_2}}{\partial h_{t_1}} = \prod_{t=t_1}^{t_2} \frac{\partial h_t}{\partial h_{t-1}} = \prod_{t=t_1}^{t_2} 1 = 1 \quad (z_t \rightarrow 0)$$

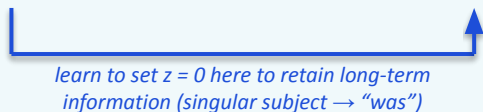
When $z > 0$, incorporate a new hidden state $\tilde{h}_t$, e.g. similar to a normal RNN.

No gate at all: a residual connection

$$h_t = h_{t-1} + \tilde{h}_t$$

Example

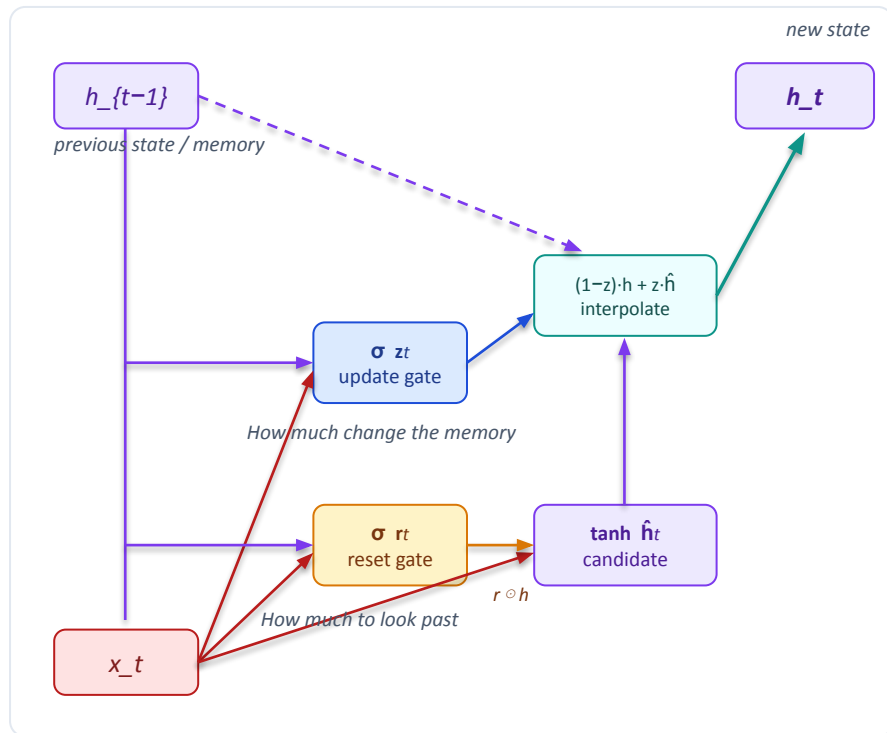
The **cat**, which sat on the mat, **was** hungry



$z \approx 0$ inside the clause: the gated state carries the subject’s features (singular, animate) unchanged, so the verb agrees with “cat”.

Putting it all together: Gated Recurrent Unit (GRU)

Cho et al. 2014, "Learning Phrase Representations using RNN Encoder-Decoder" — a 2-gate architecture



Update gate: should I update the previous hidden state?

$$z_t = \sigma(W_z x_t + U_z h_{t-1})$$

Reset gate: should I use the hidden state in the update?

$$r_t = \sigma(W_r x_t + U_r h_{t-1})$$

Candidate state: a normal RNN update on the reset state

$$\hat{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}))$$

Recurrent update: interpolate between old and candidate

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \hat{h}_t$$

GRU step by step: what each gate decides

1 Reset gate r_t

How much of the previous state may be used to propose new content? $r \approx 0$ means “forget the past when proposing”.

2 Candidate $\hat{h}_t$

A normal RNN update computed on the reset state $r \odot h_{t-1}$:
What is the proposed new memory.

3 Update gate z_t

How much of the state should be overwritten? $z \approx 0$ means “keep what I have”; $z \approx 1$ means “replace it”.

4 Interpolate

$h_t = (1-z) \odot h_{t-1} + z \odot \hat{h}_t$ — a soft, per-dimension blend of old memory and new proposal.

The extremes show what the gates buy you

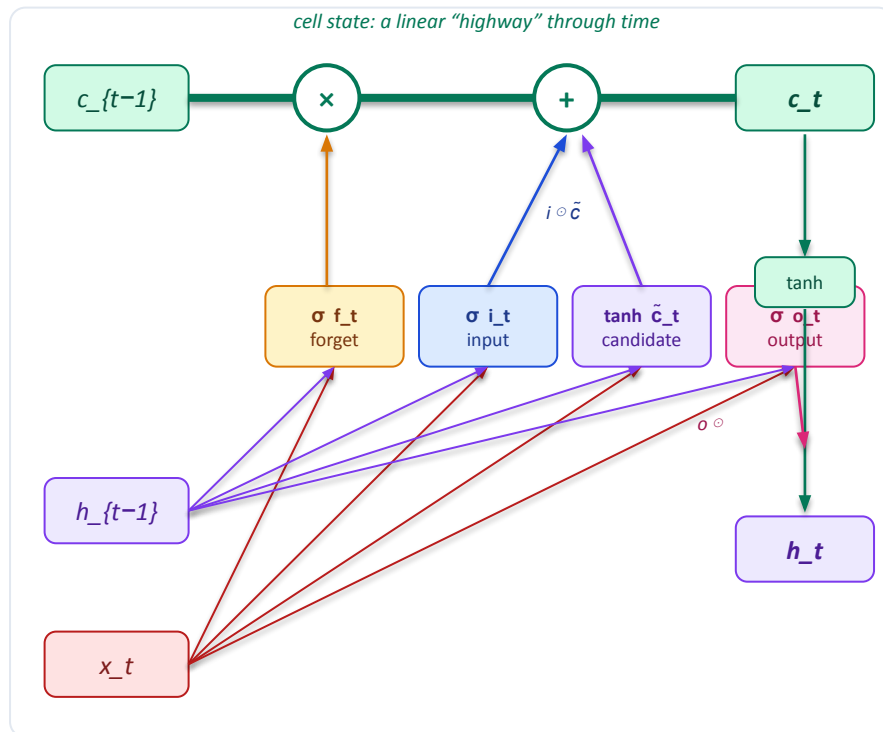
$z_t = 0$: $h_t = h_{t-1}$ (perfect copy; gradient exactly 1)

$z_t = 1, r_t = 1$: $h_t = \tanh(W_h x_t + U_h h_{t-1})$ (a vanilla RNN)

$r_t = 0$: $\hat{h}_t = \tanh(W_h x_t)$ (ignore the past, start fresh)

Long Short-Term Memory (LSTM)

Hochreiter & Schmidhuber 1997 — a 3-gate architecture using an additional context (“cell”) vector c



Forget gate: should I update (erase) the previous context?

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

Input gate: how much of the new candidate to write?

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

Candidate: new content, from the input and previous h

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

Cell update: additive! gradients flow through the $+$ path (**Long Term Memory**)

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Output gate / hidden: what to reveal from the cell as h_t (**Short term memory**)

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

Intuition (StatQuest): the cell state is the long-term memory, h is the short-term memory; gates are percentages in $[0,1]$.

LSTM in plain words: long-term and short-term memory

Two memories: the cell state c is a **long-term memory** that passes straight through time; the hidden state h is a **short-term memory** recomputed every step and used for the output. Each gate is a vector of percentages in $[0, 1]$.

Stage 1 · Forget

What % of the long-term memory do we keep?

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

$$c \leftarrow f \odot c_{t-1}$$

$f = 1$ keeps everything, $f = 0$ wipes the memory. Usually initialised with a positive bias so the cell starts by remembering.

Stage 2 · Input

What % of the new candidate do we add?

$$i_t = \sigma(\dots), \tilde{c}_t = \tanh(\dots)$$

$$c \leftarrow c + i \odot \tilde{c}$$

Two parts: the candidate $\tilde{c}$ (what could be written, in $[-1, 1]$) and the input gate i (how much of it to write).

Stage 3 · Output

What % of the memory do we reveal now?

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

$$h_t = o \odot \tanh(c_t)$$

The new short-term memory h_t is a gated view of the long-term memory; it feeds the output layer and the next step.

Framing follows StatQuest, "Long Short-Term Memory (LSTM), Clearly Explained"; equations as in Hochreiter & Schmidhuber (1997).

Bidirectional RNNs (and BiLSTMs)

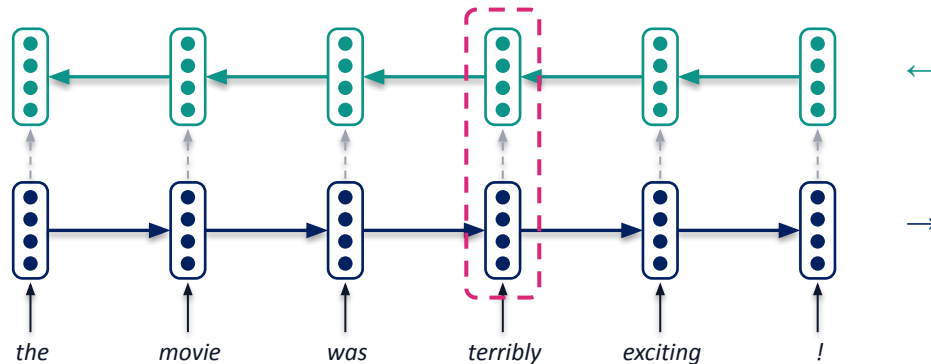
Motivation: sentiment classification

the movie was **terribly** exciting !

A left-to-right RNN reads “terribly” before “exciting”, so its state for “terribly” only knows the **left context** (negative!). The right context would flip it to positive.

Forward RNN ($\rightarrow$) + Backward RNN ($\leftarrow$), concatenated

dashed: concatenated state for “terribly”



General notation

$$\vec{h}^{(t)} = \text{RNN}_{FW}(\vec{h}^{(t-1)}, x^{(t)})$$

$$\overleftarrow{h}^{(t)} = \text{RNN}_{BW}(\overleftarrow{h}^{(t+1)}, x^{(t)})$$

$$h^{(t)} = [\vec{h}^{(t)}; \overleftarrow{h}^{(t)}]$$

Two separate RNNs with **separate weights**; the concatenated state is the “contextual representation” of word t , seeing both sides.

- Only applicable if you have access to the **entire input sequence** — so **not for language modeling**, where only left context is available.
- For any kind of **encoding** (tagging, classification, the NMT encoder) bidirectionality is powerful: **use it by default**. BiLSTM taggers and ELMo are built this way.
- BERT** = Bidirectional Encoder Representations from Transformers: a pretrained contextual representation system built on bidirectionality (with attention instead of recurrence — Sessions 6–7).

6.

Machine Translation: from Statistical MT to Neural MT

Machine Translation: the task and its history

Machine Translation (MT)

...is the task of translating a sentence x from one language (the **source language**) to a sentence y in another language (the **target language**).

x : *I like deep learning*

y : 我喜欢深度学习

1950s

Early MT: rule-based

Began in the early 1950s on machines, driven by the Cold War: Russian $\rightarrow$ English. Systems were mostly rule-based, using a bilingual dictionary to map Russian words to their English counterparts.

1990s–2010s

Statistical MT (SMT)

Core idea: learn a probabilistic model from data. Decompose $P(y | x)$ with Bayes' rule into a translation model learnt from parallel text and a language model learnt from monolingual text.

2014

Neural MT (NMT)

A single end-to-end neural network: the sequence-to-sequence model with two RNNs. From a fringe attempt in 2014 to the standard method by 2016.

2017–

Transformers

Attention replaces recurrence entirely (Vaswani et al. 2017); the architecture behind today's LLMs. Next session.

SMT: learn a probabilistic model from data

Suppose we translate French $\rightarrow$ English. We want to find the best English sentence y given the French sentence x :

$$\arg \max_y P(y|x) \quad \text{use Bayes' rule}$$

Why Bayes? Splitting $P(y | x)$ into two factors lets each model be learnt from the data it needs — and lets the LM be trained on huge monolingual corpora.

Translation model $P(x | y)$

- Models how words and phrases should be translated — **fidelity**
- Learnt from **parallel data**: pairs of human-translated French/English sentences

Language model $P(y)$

- Models how to write good English — **fluency**
- Learnt from **monolingual data** — far more plentiful than parallel data
- **This is exactly the language model of Part 1** (an n-gram LM in classic SMT systems).

Learning the translation model: parallel data and alignment

Question: how do we learn $P(x | y)$? First, we need a large amount of **parallel data** (e.g. the Rosetta Stone). Then we break the problem down further by introducing a latent variable a , the **alignment**: the word-level correspondence between x and y .

One-to-one, plus a “spurious” word (Le has no counterpart)

$$P(x, a | y)$$

model the French sentence and its alignment to the English jointly

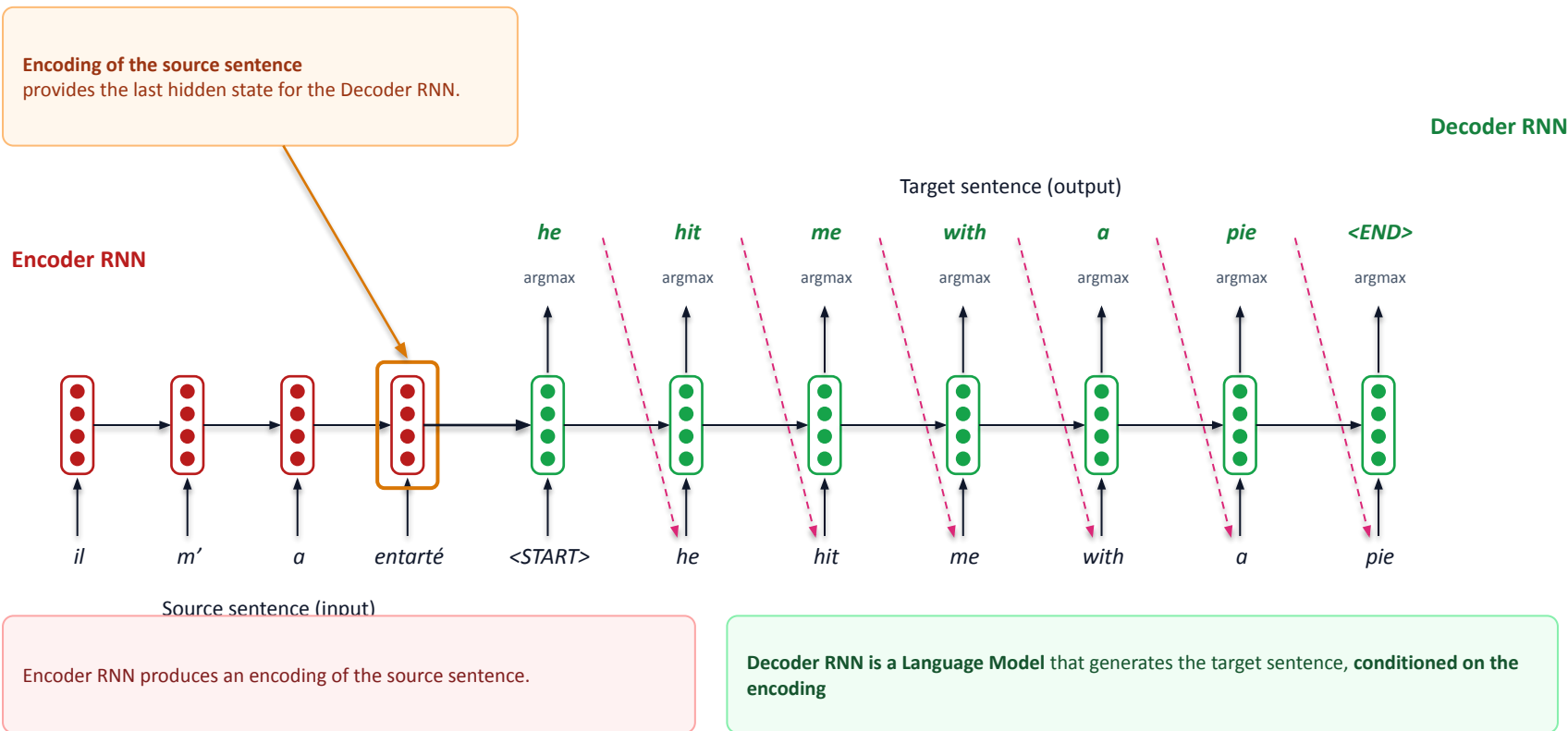
		shaken	two	quakes	
	Japan	by	new		
Le					
Japon					
secoué					
par					
deux					
nouveaux					
séismes					

One-to-many: “implemented” is a fertile word

		program	been	
	The	has	implemented	
Le				
programme				
a				
été				
mis				
en				
application				

- Alignment is the correspondence between particular words in the translated sentence pair. It can be **one-to-one**, **many-to-one**, **one-to-many** (“fertile” words such as implemented → mis en application) or **many-to-many** (phrase level: “don’t have any money” ↔ “sont démunis”).
- Some words have no counterpart at all: **spurious words** (the French article “Le” above). Typological differences between languages lead to complicated alignments.

Neural Machine Translation: the sequence-to-sequence model



Seq2seq is a conditional Language Model — trained end-to-end

The sequence-to-sequence model is an example of a Conditional Language Model

- **Language Model** because the decoder is predicting the next word of the target sentence y
- **Conditional** because its predictions are *also* conditioned on the source sentence x

NMT directly calculates $P(y | x)$:

$$P(y|x) = P(y_1|x)P(y_2|y_1,x)P(y_3|y_1,y_2,x)\cdots P(y_T|y_1,\dots,y_{T-1},x)$$

Probability of next target word, given target words so far and source sentence x

Training: a loss at every decoder step

$$J = \frac{1}{T} \sum_{t=1}^T J_t = \frac{1}{T} (J_1 + J_2 + J_3 + J_4 + J_5 + J_6 + J_7)$$

$J_1 = -\log P(\text{"he"}), J_4 = -\log P(\text{"with"}), J_7 = -\log P(\text{<END>})$

$$\min_{\theta} \sum_{(x,y) \in D} \sum_t -\log p_{\theta}(y_t | y_{<t}, x)$$

Seq2seq is optimized as a **single system**. Backpropagation operates "end-to-end".

Sequence-to-sequence is versatile!

The general notion here is an **encoder-decoder model**: one neural network takes input and produces a neural representation; another network produces output based on that representation. If the input and output are sequences, we call it a **seq2seq** model.

Summarization

long text → short text

Dialogue

previous utterances → next utterance

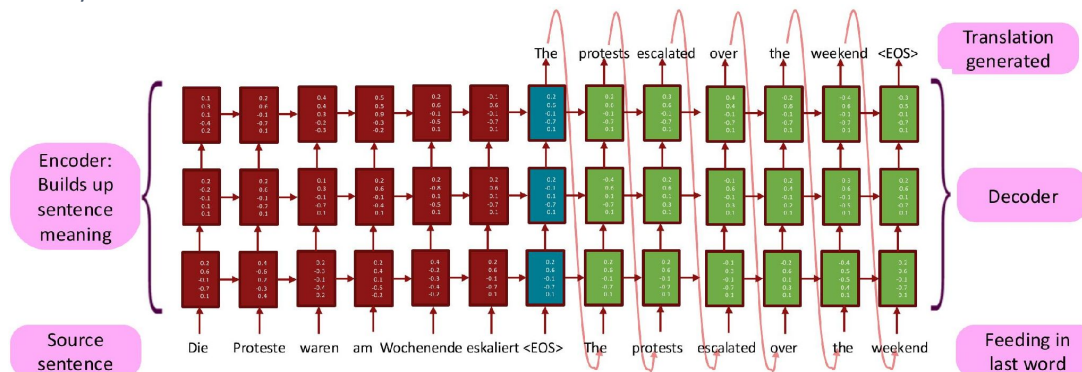
Parsing

input text → output parse as a sequence

Code generation

natural language → Python code

Multi-layer deep encoder-decoder machine translation net [Sutskever et al. 2014; Luong et al. 2015] — the hidden states from RNN layer i are the inputs to RNN layer $i+1$



NMT: the first big success story of NLP deep learning

Neural MT went from a fringe research attempt (2014) to the leading standard method (2016)

2014 First seq2seq paper published [Sutskever et al. 2014]

2016 Google Translate switches from SMT to NMT

2018 ...and by 2018 everyone has



Microsoft



SYSTRAN
beyond language



This is amazing!

SMT systems, built by **hundreds** of engineers over many **years**, outperformed by **NMT** systems trained by **small groups** of engineers in a few **months**.

Why now? Everything we built in Parts 2–5 — an RNN that reads any-length input, trained end-to-end by BPTT, with gated cells that remember — is exactly what NMT needed: **a language model conditioned on another sentence**.

Advanced NLP - Session 5 - Recurrent Sequence Architectures: RNNs, LSTMs & GRUs - IUST

Language Models

RNNs

Training RNNs

Vanishing Gradients

GRU & LSTM

Machine Translation